SHOP-CO / KAMPUNGCETAK

SYSTEM REVIEW

Frontend & Backend Report

Features, known bugs, order progress, operating steps, system mechanics, and a simple record of fixes committed to GitHub.

Prepared
30 July 2026

Format
A4 portrait, print ready

Repository
github.com/iniharith/shop-co

Reviewed revision
`4c997f5c` on `main`

Prepared from a source-code and Git-history review. Findings describe the repository state at the reviewed revision and are not a production penetration test.

OVERVIEW

Executive Summary

4

User interfaces: customer web, admin web, admin mobile, and admin desktop wrapper.

1

Express API backed by MongoDB, Redis, Socket.IO, S3, EasyParcel, and messaging services.

Main order-flow finding: the storefront currently displays products from a local dummy catalog and stores those items in browser `localStorage`. Backend order creation reads only the authenticated user's MongoDB cart. As a result, a normal checkout using the displayed catalog can reach an empty-cart error instead of creating an order.

Current system position

Area	What is working	Current limitation
Storefront	Responsive product discovery, rich print configuration, authentication, cart UI, profile, artwork upload, orders, chat, and tracking screens.	The active catalog/cart path is partly mocked and does not reliably feed backend order creation.
Admin workflow	Order, task, artwork, production, packaging, shipping, user, chat, and system-status tools across web and mobile.	Some endpoints need stronger authorization and several mobile performance fixes are still being refined.
Backend	Persistent data, token authentication, cache/pub-sub, task synchronization, file handling, shipment booking, and notification integrations.	Security hardening, transactional stock handling, cache invalidation, and test coverage need priority.
Delivery	EasyParcel quotation, booking, AWB, webhook/cron tracking, and customer tracking timeline are implemented.	Web COD orders remain `PENDING` while shipment creation requires `PAID`.

Recommended priority

1. Connect the visible product configurator and cart to one authoritative backend product/order model.
2. Protect order, task, chat, folder, seed, and migration endpoints with ownership or staff-role checks.
3. Fix payment-to-shipping progression and make stock/order writes transactional.
4. Correct customer progress mapping and expose the real production statuses.
5. Add integration tests for browse-to-order, artwork review, production, shipment, and delivery.

Features & Mechanics

1. Frontend features

Customer shopping

- Homepage, categories, featured items, search, pagination, and responsive filters.
- Configurable print products with material, format, finishing, add-ons, turnaround, quantity, and matrix pricing.
- Authenticated cart, address checkout, order history, profile editing, and delivery tracking.
- Artwork upload through S3 presigned URLs with validation, notes, history, and deletion.
- Customer support chat, English/Malay localization, dark mode, and responsive layouts.

Staff interfaces

- Next.js admin dashboard for orders, tasks, artwork, production, packaging, files, users, and shipping.
- Expo mobile admin app covering orders, tasks, production, packaging, tracking, chat, files, and server health.
- Electron desktop wrapper for the deployed administration site.
- RSVP invitation experience with countdown, music, maps, wishes, and attendance controls.

5. How the frontend works

1. Render
Next.js App Router renders public and protected pages with React 19 components.

2. Session
NextAuth stores the backend JWT in the user session and protects cart/profile routes.

3. Data
Axios calls the API; TanStack Query caches server responses and refreshes UI state.

4. Updates
Socket.IO and polling support task, notification, file, chat, and shipment updates.

Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · React Query · NextAuth · Zustand · Socket.IO · S3 direct upload

> **Important implementation detail:** products whose IDs start with `prod-` use the local dummy catalog and browser cart. Other products use the backend API. This split behavior is the source of several checkout inconsistencies.

Primary evidence: `frontend/src/app/page.tsx`, `frontend/src/hooks/useProducts.ts`, `frontend/src/api/cart.ts`, `frontend/src/components/page-sections/shop/product-details.tsx`, `frontend/src/config/auth.config.ts`, and `frontend/src/utils/s3Upload.ts`.

Order Progress & How to Use

3. Frontend order progress

The intended customer journey is:

Browse	Configure	Order	Follow
Search, filter, and open a print product.	Select design, material, size, finishing, quantity, and turnaround.	Log in, add to cart, provide an address, and submit COD checkout.	Upload artwork, review comments, and track production and delivery.

The customer UI currently summarizes progress as:

```
ORDER PLACED -> PROCESSING -> SHIPPED -> DELIVERED
```

The delivery tracker adds:

```
PENDING -> PICKED UP -> IN TRANSIT -> OUT FOR DELIVERY -> DELIVERED
```

The backend has many more artwork, design, printing, and packaging states. The customer progress component does not yet represent them correctly.

4. Frontend steps: how to use

1. Open `https://studioivory.art/` and choose **Shop**.
2. Use search or filters to locate a product, then open its detail page.
3. Select all printing options. Choose whether to upload artwork or request design service.
4. Sign in or register, then select **Add to cart**.
5. Open the cart, review items and quantities, and continue to checkout.
6. Enter customer and Malaysian delivery details, or use the saved profile address.
7. Submit the order. Current web checkout records the payment method as COD.
8. Open **Profile > Upload**, choose the order, request an upload URL, and send the artwork.
9. Use **Profile > Orders** to read task comments and view tracking updates.

> **Current user warning:** checkout with the displayed `prod-` catalog may fail because those cart items remain in the browser instead of the backend cart. Do not treat the success of the cart screen as confirmation that the server can create the order.

Primary evidence: `frontend/src/app/home/shop`, `frontend/src/app/home/cart`, `frontend/src/app/home/profile/upload/page.tsx`, `frontend/src/components/page-sections/profile/orderTracker.tsx`, and `frontend/src/app/home/profile/orders/[id]/page.tsx`.

Known Frontend Bugs

These findings are based on source inspection. “Critical” means the issue can block the main purchase path; “High” means security or major business behavior is affected.

Level	Issue	Simple explanation
Critical	Catalog/cart disconnect	Displayed products are local dummy records. Their cart items are saved in the browser, while order creation reads the database cart, so checkout can fail as empty.
Critical	Configuration not preserved	The configurator calculates detailed prices, but selected print options and calculated totals are not sent as structured order data. A fake artwork URL is used.
High	Incorrect progress mapping	At `SHIPPED` and `DELIVERED`, the completion logic is off by one; cancellation can appear as all positive steps completed.
High	Size string mismatch	The UI appends design text to the size. The backend looks for that exact value in inventory sizes and can reject a valid product as out of stock.
Medium	Payment claims exceed behavior	A public page advertises cards, FPX, and e-wallets, but checkout creates COD orders only.
Medium	Silent cart failures	API errors fall back to local cart data, and clear-cart can report success even when the backend request failed.
Medium	Missing real-time provider	The customer Socket provider exists but is not mounted in the root provider tree.
Medium	Incomplete controls	Material and turnaround filters are displayed but not applied; several footer/dashboard links have no page.
Medium	Checkout and address UX	The submit spinner watches cart loading rather than order creation; profile address mapping can replace the address with the state value.
Medium	Build/type risk	Production builds ignore TypeScript errors, allowing integration mistakes to ship.

Evidence includes `frontend/src/hooks/useProducts.ts:39-68`, `frontend/src/api/cart.ts:23-77`, `frontend/src/components/page-sections/shop/product-details.tsx:69-80`, `frontend/src/app/home/profile/orders/[id]/page.tsx:44-69`, `frontend/src/components/provider/index.tsx`, `frontend/src/components/forms/addressForm.tsx:73-89`, and `frontend/next.config.ts:39-41`.

Fixes Committed to GitHub

Recent frontend and admin-interface fixes, rewritten in simple language. Commit IDs link the report back to Git history.

`4c997f5c`	30 Jul 2026	**Mobile task list:** reduced the amount of work needed to display tasks on phones.
`d427a3a7`	30 Jul 2026	**iPad stability:** limited task queries and used lighter animations on touch devices to reduce crashes.
`3efb0a6f`	29 Jul 2026	**Task form typing:** removed performance work that caused lag while users typed.
`5fa0cf89`	28 Jul 2026	**Project files:** added reliable range selection for choosing multiple files.
`40555066`	27 Jul 2026	**Mobile memory:** reduced hero image size and adjusted loading to lower memory use.
`53ccc2d5`	27 Jul 2026	**Admin mobile layout:** corrected page overflow, improved Safari compatibility, and added a socket polling fallback.
`6b28e10d`	27 Jul 2026	**Authentication:** removed an unsafe fallback cookie and corrected middleware type handling.
`f4cd4927`	24 Jul 2026	**Share previews:** public file links now show the shared file instead of the admin console.
`785de2ab`	22 Jul 2026	**Production navigation:** corrected movement from Production to Packaging and from Packaging to Shipped.
`10110dd1`	22 Jul 2026	**Expired sessions:** forced logout and prevented an endless token-refresh loop.

> **Interpretation:** recent work has focused heavily on admin/mobile stability, artwork management, authentication, and workflow navigation. The customer storefront cart/backend integration remains the most important unfinished frontend item found in this review.

Repository: `https://github.com/iniharith/shop-co`. Commit list inspected through `4c997f5c` on 30 July 2026.

Features & Mechanics

1. Backend features

Commerce and workflow

- Registration, login, refresh tokens, profiles, roles, products, carts, and orders.
- Order-linked staff tasks, assignment, comments, activities, status synchronization, and notifications.
- Manual Shopee, TikTok, and web orders plus production and packaging queues.
- EasyParcel quotation, shipment booking, AWB storage, tracking webhooks, and scheduled reconciliation.

Files and communication

- S3 presigned uploads, artwork review, virtual folders, share links, ZIP streaming, and download progress.
- Redis caching and pub/sub with Socket.IO client/admin namespaces.
- WhatsApp Cloud API, incoming support messages, customer status updates, and Expo push notifications.
- Audit logs, server status, project files, and local image-upscaling tools.

5. How the backend works

Request
Express routes validate JWTs and pass work to controllers/use cases.

Business logic
Use cases calculate orders, synchronize tasks, and coordinate external services.

Persistence
Mongoose repositories save users, carts, products, orders, tasks, files, and parcels.

Distribution
Redis, Socket.IO, WhatsApp, Expo, and cron jobs distribute changes and reconcile state.

Node.js · Express 4 · TypeScript · MongoDB · Redis · Socket.IO · AWS S3 · EasyParcel · WhatsApp

Architecture: routes and controllers form the HTTP layer; application use cases hold business rules; repositories/models persist data; infrastructure services integrate storage, shipping, messaging, sockets, caching, and jobs.

Primary evidence: `backend/src/settings/app.ts`, `backend/src/settings/server.ts`, `backend/src/application/usecases`, `backend/src/infrastructure`, and `backend/src/presentation/routes`.

Order Progress & How to Use

3. Backend order progress

```
PLACED -> IN_PROGRESS -> PENDING_ARTWORK -> ARTWORK_REVIEWED / ARTWORK_REJECTED -> IN_DESIGN -> PEMBETULAN ->
DONE_DESIGN -> IN_PRODUCTION -> HOLD_PRINTING / DONE_PRINTING -> PACKAGING -> SHIPPED -> IN_TRANSIT -> DELIVERED
```

Alternative terminal states include `RETURNED`, `CANCELLED`, and `FAILED`.

1. `POST /api/orders` loads the authenticated user's MongoDB cart and validates products, sizes, and stock.
2. The backend recalculates price, decreases stock, creates a COD order, sends a notification, clears caches/cart, and creates a linked task.
3. Staff move the task through artwork, design, printing, and packaging. Task and order status are synchronized.
4. For a paid order in `DONE_PRINTING` or `PACKAGING`, staff request an EasyParcel quotation and book shipment.
5. Shipment and AWB details are saved. Webhook and 15-minute cron updates move the parcel through transit to delivery.
6. On delivery, the current code removes linked task artwork from S3 and file records.

4. Backend steps: how to use the API

Purpose	Request	Notes
Health	`GET /health`	Confirms the API process is available.
Session	`POST /api/auth/register` `POST /api/auth/login`	Send protected calls with `Authorization: Bearer <token>`.
Products	`GET /api/products`	Also supports search, category, filtering, and product detail.
Cart	`POST /api/cart/add` `GET /api/cart`	Add `productId`, `size`, `quantity`, and optional artwork URL.
Order	`POST /api/orders` `GET /api/orders/user`	Create from the server cart; list the signed-in user's orders.
Artwork	`POST /api/files/presigned-url` `POST /api/files/save-metadata`	Upload bytes directly to S3 between these calls.
Shipping	`POST /api/orders/:id/shipping/quotations` `POST /api/orders/:id/ship`	Staff only; requires payment and a shippable production status.

Known Backend Bugs & Risks

Level	Issue	Simple explanation
Critical	Missing authorization	Authenticated users can reach several order, task, chat, and folder operations without consistent ownership or staff-role checks.
Critical	Public data-changing routes	Product reseeding and status migration routes can modify database data without authentication.
Critical	Tracked credentials	Compose/environment artifacts contain secrets or credential-like values. Secrets should be rotated and removed from Git history.
High	Non-transactional stock	Stock only checks for greater than zero, not requested quantity, and is reduced before order creation without a database transaction.
High	Payment blocks shipment	Web orders are COD/PENDING, while EasyParcel shipment creation requires PAID. No normal payment-completion path was found.
High	Wrong status query field	The repository searches `status`, but the schema stores `orderStatus`, so status queues can return no results.
High	Socket impersonation risk	The customer socket trusts a query-string user ID and does not use the JWT middleware applied to admin sockets.
High	Open CORS and webhook trust	CORS accepts every origin with credentials, and incoming WhatsApp webhooks are not signature verified.
Medium	Stale status caches	Status-specific order lists can remain cached for up to 24 hours after an order moves.
Medium	Destructive delivery cleanup	Marking an order delivered permanently deletes linked artwork rather than applying a retention/archive policy.
Medium	Error handling and tests	Some authentication failures become HTTP 500; process-level exceptions are ignored; no automated test suite is configured.

Evidence includes `backend/src/presentation/routes/order.route.ts`, `taskRoutes.ts`, `chatRoutes.ts`, `admin.route.ts`, `product.route.ts`, `backend/src/application/usecases/orders/order.usecase.ts`, `backend/src/infrastructure/db/repositories/order.repository.ts:63-65`, `backend/src/infrastructure/socket/socketHandler.ts`, and `docker-compose.yml`.

Committed Fixes & Next Actions

6. Backend fixes committed to GitHub

`e6df6c9a`	22 Jul 2026	**MongoDB grouping:** corrected regular-expression and ObjectId handling in artwork folder queries.
`f744162d`	22 Jul 2026	**Admin CORS:** corrected the production admin origin problem.
`5c0f2bda`	22 Jul 2026	**Cache resilience:** added an in-memory fallback when Redis folder caching is unavailable.
`eff074ca`	22 Jul 2026	**Redis stability:** reduced reconnect storms and made cache reads non-blocking.
`ac1a512a`	22 Jul 2026	**File-query speed:** reduced repeated database work with projections, batching, and caching.
`3cb1599a`	22 Jul 2026	**Shipping tracking:** completed the EasyParcel tracking workflow.
`d1ff1bab`	21 Jul 2026	**Customer shipping updates:** made shipment notifications safer and more reliable.
`773b1d24`	21 Jul 2026	**Artwork uploads:** stabilized public artwork upload behavior.

Recommended action plan

Priority	Action	Expected result
P0	Rotate exposed secrets; remove public mutation routes; enforce role/ownership authorization.	Immediate reduction in data and account risk.
P0	Replace the split mock/API cart with one backend-backed product configuration and order payload.	A reliable customer checkout path.
P1	Use MongoDB transactions and atomic quantity checks for stock and order creation.	No partial stock reduction or overselling.
P1	Define COD/payment completion and shipping eligibility rules.	Orders can progress into EasyParcel without manual database changes.
P1	Add API and browser integration tests for the complete order lifecycle.	Future commits detect regressions before deployment.

> **Report conclusion:** Shop-Co has broad operational capability and a mature staff workflow, especially around artwork, production, files, and shipping. The next engineering milestone should be making the customer purchase path authoritative end to end, followed immediately by authorization and secret-management hardening.

Prepared from repository source and Git history at `github.com/iniharith/shop-co`. Report source: `reports/shop-co-system-report.html`.